

CSE398 Lab1 Object Detection Report

Ruoxi Wang

September 2026

1 Introduction

This report is based on three object detection experiments and explains how pretrained YOLOv11 works, what was learned from fine-tuning the model on a custom nail-clipper dataset, and how YOLOE performs open-set object detection. The first experiment used pretrained YOLOv11 to detect common objects. The second experiment fine-tuned and evaluated YOLOv11 using a custom dataset of collected and annotated nail-clipper images. The third experiment used text and visual prompts to test YOLOE’s ability to detect objects from open categories. Based on these experiments, this report also examines the major architectural changes that allow YOLOE to move beyond a fixed category set and perform open-set object detection.

2 How YOLOv11 Works

YOLOv11 is a one-stage object detection model. “One-stage” means that the model predicts object locations and categories within a single forward pass, rather than first generating candidate regions and then classifying each region separately [1]. The input image is first resized to the dimensions required by the model and then processed by the neural network to extract visual features. Using these features, the model predicts multiple candidate bounding boxes and assigns an object category and confidence score to each prediction [2]. Low-confidence predictions are subsequently filtered, and duplicate boxes associated with the same object are removed through non-maximum suppression to produce the final detections [3]. This unified detection process allows the YOLO family to balance detection speed and accuracy, making it suitable for object detection in both images and real-time video [1].

The YOLOv11 network can be summarized by three main components: the backbone, neck, and detection head. The backbone extracts features from the input image, such as object edges, textures, and shapes. As the image is processed through successive network layers, these features develop from simple visual information into more abstract object representations. The neck combines features from different layers and scales, allowing the model to detect objects of different sizes within an image. Finally, the detection head uses the

combined features to predict object bounding boxes and classes. Through this division of responsibilities, YOLOv11 first processes the visual information in an image and then determines where objects are located and which categories they belong to [4].

2.1 Experiment 1: Pretrained YOLOv11 Object Detection

2.1.1 Objective

Experiment 1 required cloning the GitHub repository provided by the instructor, capturing images of everyday objects in an apartment or laboratory, and using these photographs as test images to evaluate how well pretrained YOLOv11 could recognize the objects. The purpose of this experiment was not only to run an existing object detector, but also to observe how the model produced bounding boxes, class labels, and confidence scores and to identify which objects it could or could not recognize reliably. The unsuccessful cases observed in Experiment 1 were also used to select an object for fine-tuning in Experiment 2.

2.1.2 Experimental Setup

The experiment was conducted in Google Colab using an NVIDIA Tesla T4 GPU. The provided `webcam-object-recognition` repository was first cloned, after which Ultralytics was installed and the pretrained `yolo11n.pt` model contained in the repository was loaded. The model included 80 pretrained classes [5]. Nine self-captured test images were used, containing everyday objects such as a mouse, keyboard, cup, nail clipper, medicine box, spoon, cat, hat, hair-brush, and hand cream.

During inference, the images were resized to 640×640 , the confidence threshold was set to 0.25, and the GPU was selected using `device=0`. All other inference parameters retained their Ultralytics default values. For each image, the model returned predicted classes, bounding boxes, and confidence scores.

2.1.3 Results

The pretrained YOLOv11n model produced at least one detection in eight of the nine test images, while the hand-cream image produced no detection. The model correctly recognized some common objects, including cups, keyboards, bottles, and a monitor, but it also produced clear class confusions. For example, the mouse was classified as a `vase`, the spoon as a `toothbrush`, and the cat as a `bear`. For the nail clipper, the model did not predict the correct category and instead classified it as a `knife` and a `cell phone`.

Table 1 compares the prominent objects selected through manual inspection with the model predictions. Objects visible only incidentally in the background were not included as evaluation targets. A correct detection required both an appropriate class label and a bounding box corresponding to the selected object.

Therefore, these counts provide a descriptive comparison of the nine test images rather than a formal evaluation using a fully annotated test dataset.

Table 1: Comparison between the prominent objects in the Step 1 test images and the predictions produced by pretrained YOLOv11n.

Image	Prominent objects	YOLOv11 predictions	Correct	Missed	Incorrect
WechatIMG1280	Mouse	Vase	0	1	1
WechatIMG1281	Keyboard, cup, nail clipper	Keyboard, cup	2	1	0
WechatIMG1282	Nail clipper	Knife, cell phone	0	1	2
WechatIMG1283	Cup, medicine box, monitor	Cup, book, TV	2	1	1
WechatIMG1284	Spoon	Toothbrush	0	1	1
WechatIMG1285	Cat	Bear	0	1	1
WechatIMG1286	Hat, keyboard, two bottles	Two keyboards, two bottles, cell phone, dining table	3	1	3
WechatIMG1287	Hairbrush, spoon, nail clipper	Knife, scissors	0	3	2
WechatIMG1288	Hand cream	No detection	0	1	0
Total	18 selected objects	18 predictions	7	11	11

As summarized in Table 1, seven of the 18 selected prominent objects were correctly detected, while 11 were missed. Among the 18 predictions produced by the model, 11 were incorrect. These results indicate that the pretrained model could directly recognize some familiar categories but could miss or misclassify objects not represented in its pretrained category set or objects that visually resembled known categories. The failure to recognize the nail clipper was especially relevant because it appeared in several test images and was instead associated with incorrect categories. Therefore, the nail clipper was selected as the custom class in Experiment 2 to evaluate whether fine-tuning could improve its detection.

3 YOLOv11 Fine-Tuning and Lessons Learned

3.1 Experiment 2: Fine-Tuning YOLOv11

3.1.1 Objective

Experiment 2 required creating or finding a small dataset of 50 images containing an object that pretrained YOLOv11 could not recognize correctly in Experiment 1. The model was then fine-tuned by following the provided Colab workflow, and the selected object was tested again to determine whether fine-tuning improved detection. Based on the results of Experiment 1, the nail clipper was selected as the custom detection class. The pretrained model did not include a **nail clipper** category and incorrectly classified the nail clipper as a **knife** and a **cell phone** in the test image. Therefore, the main objective of Experiment 2 was to use bounding-box annotations to teach the model this new category and compare its detection performance before and after fine-tuning.

3.1.2 Dataset and Experimental Setup

I captured 50 nail-clipper images with variations in viewing angle, object position, and background. The images were uploaded to Roboflow, where a bounding box was drawn around each nail clipper and assigned the class label **nail_clipper**. The dataset was then exported in YOLOv11 format and divided, using a random seed of 42, into 40 training images, five validation images, and five test images. The training set was used to update the model parameters, the validation set was used to evaluate the model during training and select the best weights, and the test set was used to observe performance on images not used for training.

The pretrained **yolo11s.pt** weights were used to initialize the model, which was trained for 10 epochs on an NVIDIA Tesla T4 GPU in Google Colab. The image size for training and validation was set to 640×640 , while the remaining training parameters retained the Ultralytics defaults. After training, the weights with the best validation performance were saved as **best.pt**. These weights were subsequently used for the final validation and for prediction on the five test images with a confidence threshold of 0.25.

3.1.3 Results and Lessons Learned

The best fine-tuned model achieved a precision of 0.989, a recall of 1.000, an mAP@0.5 of 0.995, and an mAP@0.5:0.95 of 0.995 on the five validation images. Before fine-tuning, the pretrained model did not have a **nail_clipper** category. It classified the nail clipper as a **knife** and a **cell phone** in Experiment 1. Another example of class confusion occurred when a spoon was classified as a **toothbrush**. After fine-tuning, the model successfully used the newly learned **nail_clipper** label to detect the target object in all five test images.

This comparison shows that fine-tuning enabled YOLOv11 to recognize a custom object that was not included in its original category set. However, some

test images contained additional or overlapping `nail_clipper` boxes. For example, the model produced multiple predictions for a single nail clipper in four of the five test images. Therefore, fine-tuning clearly improved class recognition, but the results were not completely free from localization errors or duplicate detections. In addition, because the validation set contained only five images collected under conditions similar to the training data, the high validation metrics should not be interpreted as evidence of broad generalization.

4 Open-Set Object Detection

Traditional closed-set object detectors can predict only the categories predefined during training. For example, the YOLOv11 model used in Experiment 1 was pretrained on 80 COCO categories and could predict these known classes, but it could not directly produce the label `nail_clipper`, which was absent from its category list. In contrast, open-set object detection aims to move beyond a fixed category set by allowing a model to detect new object categories based on information provided at inference time. In YOLOE, which was used in this experiment, a user can specify a category name through a text prompt or provide a visual prompt by drawing a bounding box around an example object. The model then matches the prompt information with visual features in the image to localize objects that were not fixed as standard output categories during training.

This approach differs from the fine-tuning performed in Experiment 2. Fine-tuning requires collecting and annotating images of a new category and then updating the model parameters. Open-set detection allows the prompt to be changed without retraining the model for every new category. This provides greater flexibility, although its performance may be affected by prompt wording, object appearance, background, object scale, and the confidence threshold.

4.1 Experiment 3: YOLOE Open-Set Detection

4.1.1 Objective

Experiment 3 required following the provided Colab to study open-set object detection and using YOLOE to perform text-prompt and visual-prompt detection on images and videos. The purpose of the experiment was to observe whether YOLOE could detect a target from a user-provided text description or visual example without retraining the model. To provide a direct comparison with Experiments 1 and 2, the nail clipper was retained as the target object for the image experiments.

4.1.2 Experimental Setup

The image experiments were conducted in Google Colab using the `yoloe-v8l-seg.pt` model. The text-prompt experiment used the same nail-clipper image and

tested multiple class names and confidence thresholds, including `nail clipper`, `finger nail clipper`, `toenail clipper`, and `lever of a nail clipper`.

For visual prompting, a bounding box was manually drawn around the complete nail clipper in the source image, `WechatIMG1282.jpeg`. This visual example was then used to search for the same category in the target image, `WechatIMG1281.jpeg`. Because the nail clipper appeared relatively small in the target image, an input size of 1280 was used, and confidence thresholds of 0.01 and 0.05 were tested.

The provided example videos were retained for the video experiments. The class `suitcase` was used for text prompting, while `croissant` was used for visual prompting.

4.1.3 Results

The text-prompt experiment showed that YOLOE detection was sensitive to the wording of the prompt. At a confidence threshold of 0.02, neither `nail clipper` nor `finger nail clipper` produced a detection, whereas `toenail clipper` produced one accurately localized bounding box. When the prompt was changed to the more fine-grained phrase `lever of a nail clipper` and the threshold was reduced to 0.01, the model produced several small boxes but failed to localize the lever correctly.

Cross-image visual prompting did not identify the target reliably. At a confidence threshold of 0.01, the model produced multiple false-positive detections in the background. Increasing the threshold to 0.05 removed all detections. Differences in object scale, orientation, and background between the source and target images likely reduced the generalization of the visual prompt. In comparison, the text-prompt and visual-prompt video workflows ran successfully with the provided example materials and generated annotated output videos.

Table 2 summarizes the settings and outcomes of the YOLOE experiments.

Table 2: Summary of the text-prompt and visual-prompt YOLOE experiments.

Prompt type	Prompt or setting	Result
Text prompt	<code>nail clipper</code> , <code>conf=0.02</code>	No detection
Text prompt	<code>finger nail clipper</code> , <code>conf=0.02</code>	No detection
Text prompt	<code>toenail clipper</code> , <code>conf=0.02</code>	One accurate detection
Fine-grained text prompt	<code>lever of a nail clipper</code> , <code>conf=0.01</code>	Incorrect localization
Visual prompt	<code>conf=0.01</code> , <code>imgsz=1280</code>	Multiple false positives
Visual prompt	<code>conf=0.05</code> , <code>imgsz=1280</code>	No detection
Text-prompt video	<code>suitcase</code> , <code>conf=0.10</code>	Annotated video generated
Visual-prompt video	<code>croissant</code> , default prediction settings	Annotated video generated

5 YOLOE Architecture

YOLOE retains the image feature extraction and object localization framework of the YOLO family but changes how the model determines object categories. A traditional YOLO detector can predict only the fixed categories defined during training, whereas YOLOE can determine the target category from a text or visual prompt provided during inference. It matches the prompt information with visual features in the image so that class prediction is conditioned on the user-provided prompt rather than being completely restricted to a fixed category list. This difference was also observed in the experiments: YOLOv11 could not directly produce the `nail_clipper` category, while YOLOE could search for this object through a text prompt without retraining the model.

To support different forms of open-set detection within a unified model, YOLOE introduces three main mechanisms: Re-parameterizable Region-Text Alignment (RepRTA) for text prompts, the Semantic-Activated Visual Prompt Encoder (SAVPE) for visual prompts, and Lazy Region-Prompt Contrast (LRPC) for prompt-free detection [6].

RepRTA supports text prompting by improving the alignment between text embeddings and image-region features, allowing the model to recognize categories specified by the user. SAVPE supports visual prompting by using semantic and activation branches to extract target features from a user-selected reference region and search for similar objects in other images. LRPC supports prompt-free detection by first identifying likely object regions and then retrieving their category names from a built-in large vocabulary without relying on an additional large language model. Together, these mechanisms allow YOLOE to support text, visual, and prompt-free open-set detection within one unified framework [6].

6 Conclusion

This lab compared three approaches for detecting a custom object: pretrained detection, model fine-tuning, and open-set detection. Experiment 1 showed that pretrained YOLOv11 could directly recognize some common objects but could not correctly identify the nail clipper, which was absent from its pretrained categories. Experiment 2 demonstrated that after fine-tuning on an annotated custom dataset, the model could recognize the target as `nail_clipper`, showing that fine-tuning can adapt a pretrained detector to a new specialized category. Experiment 3 showed that YOLOE could search for new target categories through text or visual prompts without retraining, although its performance was sensitive to prompt wording, confidence thresholds, and differences between the source and target images.

Overall, fine-tuned YOLOv11 provided the most consistent class recognition for the nail clipper in this lab, but it required additional data collection and annotation. YOLOE offered greater flexibility, although its detection results were more dependent on the prompt and image conditions. The notebooks,

model weights, and related results for all three experiments are available in the following GitHub repository:

CSE398-Lab1-Object-Detection

References

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788. [Online]. Available: <https://arxiv.org/abs/1506.02640>
- [2] Ultralytics, “Object detection with ultralytics yolo,” <https://docs.ultralytics.com/tasks/detect/>, accessed: 2026-09-07.
- [3] —, “Yolo configuration,” <https://docs.ultralytics.com/usage/cfg/>, accessed: 2026-09-07.
- [4] —, “Yolo architecture explained,” <https://docs.ultralytics.com/guides/yolo-architecture/>, accessed: 2026-09-07.
- [5] —, “Ultralytics yolo11,” <https://docs.ultralytics.com/models/yolo11/>, 2024, accessed: 2026-09-07.
- [6] A. Wang, L. Liu, H. Chen, Z. Lin, J. Han, and G. Ding, “YOLOE: Real-time seeing anything,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2025/html/Wang_YOLOE_Real-Time-Seeing-Anything_ICCV_2025_paper.html